Experiment # 06

Implementation of Spatial Filtering, Smoothing and Sharpening

Objective

1. To implement convolution
2. To implement spatial filtering
3. To implement Sharpening filter

Software

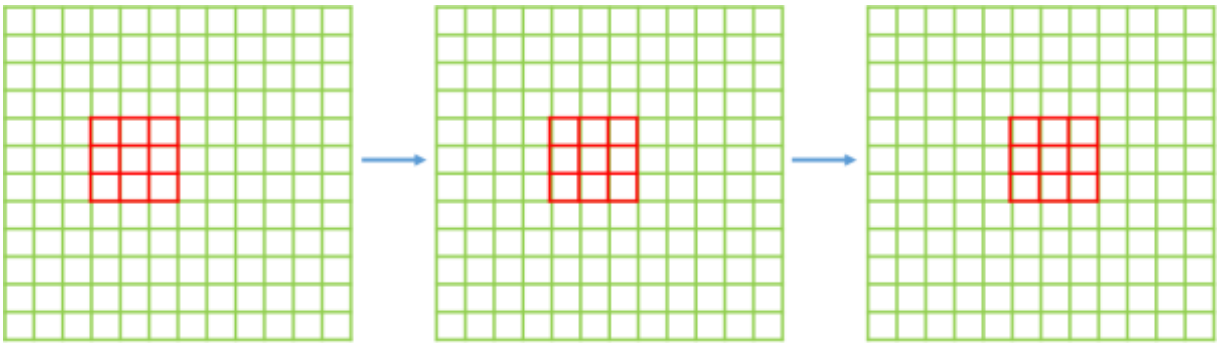
- Python Idle 3.7 or 3.8.5
- Pycharm

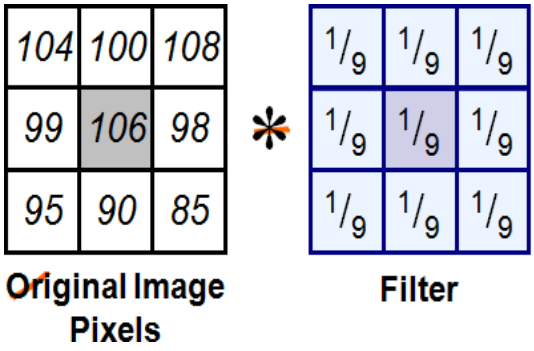
Theory

Spatial filtering is an important step of preprocessing of images. Different types of filters like smoothing filters, sharpening filters and rank filters etc. can be applied to images to achieve the desired result.

Averaging or smoothing filters consist of integration and they can be used for blurring or noise reduction. Blurring helps to remove small details from an image before further processing like object extraction can be performed on it.

Applying a smoothing filter (or any filter for that matter) involves convolving the filter of a specific size with the entire image. The mask moves pixel by pixel while convolving with the values of the pixel intensities that it covers as shown here:





$$e = 1/9*106 + 1/9*104 + 1/9*100 + 1/9*108 + 1/9*99 + 1/9*98 + 1/9*95 + 1/9*90 + 1/9*85 = 98.3333$$

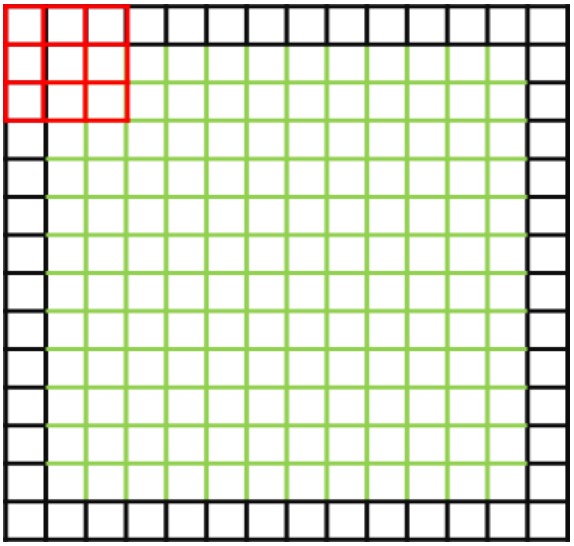
Python Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
image = cv2.imread('input_image.jpg', 0)  #Load in grayscale

# Apply Averaging Filter (Simple Blur)
average_blur = cv2.blur(image, (5, 5))  #5x5 kernel size

# Apply Gaussian Filter
gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=1)  #5x5 kernel, sigmaX=1

# Display Results
plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(average_blur, cmap='gray')
plt.title('Averaging Filter')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(gaussian_blur, cmap='gray')
plt.title('Gaussian Filter')
plt.axis('off')

plt.tight_layout()
plt.show()
```

While applying a filter to an image, the boundary of the image impose a unique challenge: for the first pixel of the first row, there are no pixels behind or above it that can be multiplied with the mask value. To counter this problem, two techniques can be used:

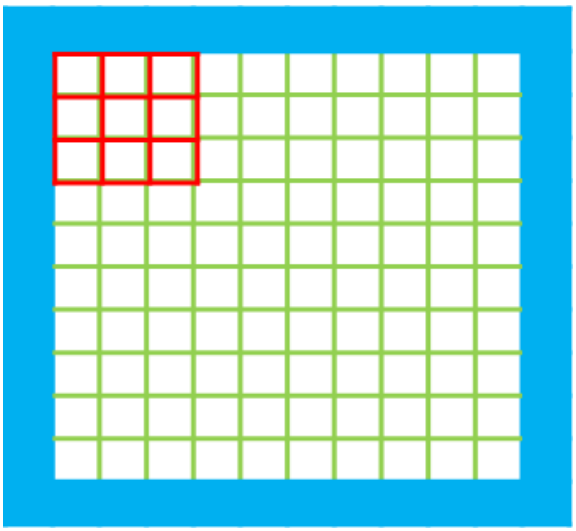
- **Padding**

  Adding a few rows of 0's all around the image helps the mask to function properly. The exact number of the 0 rows that are to be added depends on the size of the mask. OR we can replace with neighboring pixel value to avoid affect of zeros on the image



- **Ignoring some rows and columns:**

  The first few rows and columns can be ignored and the mask can be applied to the rest of the image. The number of the rows and columns that are to be ignored depends on the size of the filter.



Different spatial filters allow us to enhance the details in an image as per our requirement. Median filter can help remove salt and pepper noise, given as

$$\hat{f}(x,y) = \underset{(s,t)\in S_{xy}}{\text{median}}\{g(s,t)\}$$

Applying max and min filter can affect the boundaries of different objects in an image. For example, when max filter is applied, the boundaries of dark objects will recede while the boundaries of light objects will expand. The effect of min filter will be reverse. The formulas of min and max filters are given as

$$\hat{f}(x,y) = \max_{(s,t)\in S_{xy}} \{g(s,t)\}$$

$$\hat{f}(x,y) = \min_{(s,t)\in S_{xy}} \{g(s,t)\}$$

In order to find out the boundary of objects in an image or edge detection, Sobel filters come in handy. Sobel filter calculates the first order derivative of the image. Horizontal Sobel filter is used to find out the edges along x-axis as it calculates the derivative of an image in x direction while Vertical Sobel filter is used to find out the edges along y-axis. The edges obtained from applying both this filters can then be added to obtain all the edges in an image.

−1	0	0	−1
0	1	1	0

−1	−2	−1	−1	0	1
0	0	0	−2	0	2
1	2	1	−1	0	1

Sharpening filters enhance edges by highlighting areas with sudden intensity changes. Common sharpening filters include:

- **Laplacian Filter**: Calculates the second derivative of the image to emphasize edges.
- **Unsharp Masking**: A technique where a blurred version of the image is subtracted from the original.

0	1	0	1	1	1
1	−4	1	1	−8	1
0	1	0	1	1	1

The edges can become more pronounced as compared to Sobel filter in some cases.

Python Code

```
# Apply Laplacian Filter
laplacian_filter = cv2.Laplacian(image, cv2.CV_64F)
laplacian_filter = cv2.convertScaleAbs(laplacian_filter)  # Convert to 8-bit


# Unsharp Masking
gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=1)
unsharp_image = cv2.addWeighted(image, 1.5, gaussian_blur, -0.5, 0)
```

```
    # Display Results
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(laplacian_filter, cmap='gray')
    plt.title('Laplacian Filter')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(unsharp_image, cmap='gray')
    plt.title('Unsharp Masking')
    plt.axis('off')

    plt.tight_layout()
plt.show()
```

Some Useful Commands:

1. To sort a list: my_list_sorted = my_list.sort() OR numpy.sort(my_list)
2. To get the median of the list: my_median = numpy.median(my_list)
3. To convert a n-dimensional matrix into a vector/1D array = numpy.ravel(my_array) OR numpy.flatten(my_array)
4. To apply a filter on an image using OpenCV: filtered_image = cv2.filter2D(my_image, ddepth, mask) ddepth refers to the bit level depth of the filtered_image. As it will be the same as the source image, so this argument will be set to -1.
5. To blur image: cv2.blur(img, size), cv2.GaussianBlur(img,size,sigma), median = cv2.medianBlur(img,size)

Lab Task:

1. Write generic functions for each of the following.
    - Takes input mask size and their numeric values from the user and create the mask
    - Take image and padding size as argument and add padding on the image (either zeros or copy neighboring pixel)
    - Take image and filter as argument and apply the filter to the image
    - Take image as input and normalize the image between 0 – 255

Apply the following Masks the image and display the results

a)

1	1	1
1	1	1
1	1	1

2. Apply Horizontal Sobel and Vertical Sobel separately on the given image. Display the results. Then add the images obtained by Horizontal Sobel and Vertical Sobel together and display the resultant image with all the edges.
3. Add gaussian noise in our image through skimage.util, random noise.